SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: ITA 0605 **COURSE NAME: MACHINE LEARNING**

COURSE OUTCOME COVERED

CO1: *Learning Problems, Perspectives and Issues, Concept Learning, Version Spaces & Candidate Elimination, Inductive Bias, Decision Tree Learning, Representation & Heuristic Search (BL1, BL2)*

Assessment Tool Weightage: 40%

QUESTIONS: 15 **Total Marks: 25**

Annexure A

APPLICATION BASED QUESTIONS

Code of Conduct:

I K Harshini Sriya (192421283) *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

K Harshini Sriya

1. In Shop X, there are 20 bottles of pure oil and 30 bottles of adulterated oil, while in Shop Y, there are 40 bottles of pure oil and 20 bottles of adulterated oil. A bottle is selected at random from one of the shops, with Shop Y being twice as likely to be chosen as Shop X. The selected bottle is found to be adulterated.

Program:

```
pure_x = 20
adulterated_x = 30

pure_y = 40
adulterated_y = 20

P_X = 1 / 3
P_Y = 2 / 3

P_A_given_X = adulterated_x / (pure_x + adulterated_x)
P_A_given_Y = adulterated_y / (pure_y + adulterated_y)

P_A = (P_A_given_X * P_X) + (P_A_given_Y * P_Y)

P_X_given_A = (P_A_given_X * P_X) / P_A

print("Probability that the bottle came from Shop X =", round(P_X_given_A, 4))
```

Output:

2. A warehouse has two sections. Section A contains 15 defective items and 35 non-defective items, while Section B contains 25 defective items and 25 non-defective items. One section is chosen at random, but Section A is three times as likely to be chosen as Section B. An item selected from the chosen section is found to be defective.

Program:

```
defective_A = 15
good_A = 35
defective_B = 25
good_B = 25
P_A = 3/4
P_B = 1/4
P_D_A = defective_A / (defective_A + good_A)
P_D_B = defective_B / (defective_B + good_B)
P_D = P_D_A * P_A + P_D_B * P_B
P_A_D = (P_D_A * P_A) / P_D
print("Probability that the defective item came from Section A =", round(P_A_D, 4))
```

Output:

3. In a binary classification problem using a deep learning model with a K-Nearest Neighbour classifier, the model produces True Positive (TP) = 120 and True Negative (TN) = 150.The dataset contains 1000 samples per class, and 20% of the data is used for testing.

Program:

```
TP = 120
TN = 150
```

```
samples_per_class = 1000
test_percentage = 20
positive = samples_per_class * test_percentage // 100
negative = samples_per_class * test_percentage // 100
FN = positive - TP
FP = negative - TN
accuracy = (TP + TN) / (TP + TN + FP + FN)
precision = TP / (TP + FP)
recall = TP / (TP + FN)
specificity = TN / (TN + FP)
print("Accuracy :", round(accuracy * 100, 2), "%")
print("Precision :", round(precision * 100, 2), "%")
print("Sensitivity :", round(recall * 100, 2), "%")
print("Specificity :", round(specificity * 100, 2), "%")
```

Output:

4. A medical image classification system using a deep learning framework combined with K-Nearest Neighbour (KNN) yields TP = 180 and TN = 160. Each class contains 1200 samples, and 25% of the dataset is used as test data.

Program:

```
TP = 180
TN = 160

samples_per_class = 1200
test_percentage = 25
```

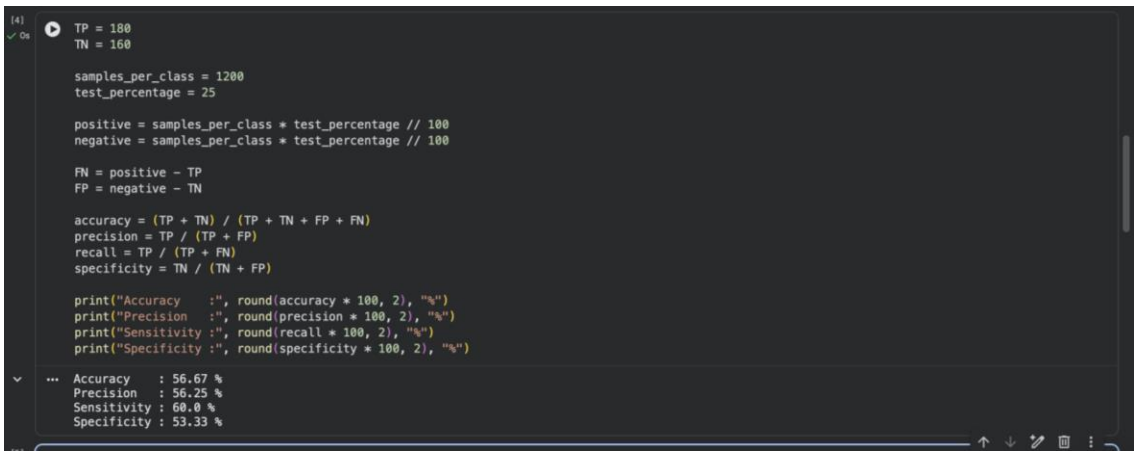
```
positive = samples_per_class * test_percentage // 100
negative = samples_per_class * test_percentage // 100

FN = positive - TP
FP = negative - TN

accuracy = (TP + TN) / (TP + TN + FP + FN)
precision = TP / (TP + FP)
recall = TP / (TP + FN)
specificity = TN / (TN + FP)

print("Accuracy :", round(accuracy * 100, 2), "%")
print("Precision :", round(precision * 100, 2), "%")
print("Sensitivity :", round(recall * 100, 2), "%")
print("Specificity :", round(specificity * 100, 2), "%")
```

Output:



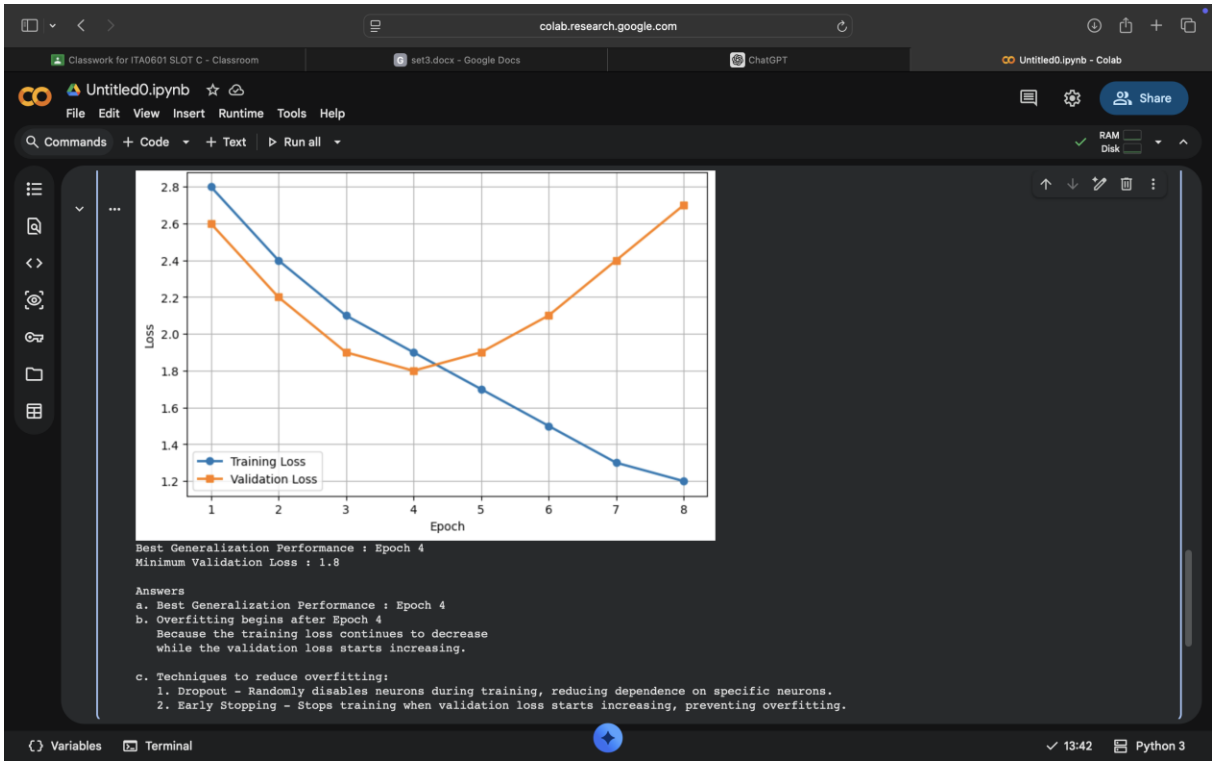
5. **A deep learning model is trained for a classification task, and the following loss values are observed:**

   i. **Epoch 1: Training Loss = 2.8, Validation Loss = 2.6**
   ii. **Epoch 2: Training Loss = 2.4, Validation Loss = 2.2**
   iii. **Epoch 3: Training Loss = 2.1, Validation Loss = 1.9**
   iv. **Epoch 4: Training Loss = 1.9, Validation Loss = 1.8**
   v. **Epoch 5: Training Loss = 1.7, Validation Loss = 1.9**
   vi. **Epoch 6: Training Loss = 1.5, Validation Loss = 2.1**
   vii. **Epoch 7: Training Loss = 1.3, Validation Loss = 2.4**
   viii. **Epoch 8: Training Loss = 1.2, Validation Loss = 2.7**

Program:

```
import matplotlib.pyplot as plt
epochs = [1,2,3,4,5,6,7,8]
training_loss = [2.8,2.4,2.1,1.9,1.7,1.5,1.3,1.2]
validation_loss = [2.6,2.2,1.9,1.8,1.9,2.1,2.4,2.7]
plt.figure(figsize=(8,5))
plt.plot(epochs, training_loss, marker='o', linewidth=2, label="Training Loss")
plt.plot(epochs, validation_loss, marker='s', linewidth=2, label="Validation Loss")
plt.title("Training Loss vs Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.xticks(epochs)
plt.grid(True)
plt.legend()
plt.show()
best_epoch = validation_loss.index(min(validation_loss)) + 1
print("Best Generalization Performance : Epoch", best_epoch)
print("Minimum Validation Loss :", min(validation_loss))
print("\nAnswers")
print("a. Best Generalization Performance : Epoch", best_epoch)
print("b. Overfitting begins after Epoch", best_epoch)
print(" Because the training loss continues to decrease")
print(" while the validation loss starts increasing.")
print("\nc. Techniques to reduce overfitting:")
print(" 1. Dropout - Randomly disables neurons during training, reducing dependence on specific
neurons.")
print(" 2. Early Stopping - Stops training when validation loss starts increasing, preventing
overfitting.")
```

Output:


colab.research.google.com
Classwork for ITA0601 SLOT C - Classroom
set3.docx - Google Docs
ChatGPT
Untitled0.ipynb - Colab
Untitled0.ipynb
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all
RAM
Disk
Loss
Epoch
Training Loss
Validation Loss
Best Generalization Performance : Epoch 4
Minimum Validation Loss : 1.8
Answers
a. Best Generalization Performance : Epoch 4
b. Overfitting begins after Epoch 4
Because the training loss continues to decrease
while the validation loss starts increasing.
c. Techniques to reduce overfitting:
1. Dropout - Randomly disables neurons during training, reducing dependence on specific neurons.
2. Early Stopping - Stops training when validation loss starts increasing, preventing overfitting.
Variables
Terminal
13:42
Python 3